



Web Technologies – SEF23

Homework 03

Hospital Management System

(with Serialization and Database Integration)

27th September, 2025 – Professor Shuja Ur Rehman Baig

Important Instructions:

- Ensure proper indentation, documentation, clean code, and well-structured classes in separate files (Follow n tier architecture)
- All functions should be in a running state without syntax errors.
- Handle all necessary exceptions and validation checks carefully in each function; otherwise, a zero grade will be assigned.
- Creating logic instead of relying on built-in functions is highly recommended.
- Always use Parameterized Queries in order to avoid SQL Injection
- Any instance of cheating or plagiarism will result in a **F grade**.

Deadline:

You must submit your assignment before **Tuesday, Sep 30, 2025 at 11:59 pm PKT**.

Contents and Objectives:

- Design and implement a simple Hospital Management System.
- Demonstrate the use of classes, data members, functions, object serialization, ADO.NET for database operations, and base object class.

Tools and Technologies:

- Visual Studio (or any preferred IDE for C# development)
- C# programming language
- .NET
- SQL Server
- ADO.NET for database operations

Interface:

Class Doctor

- **Private int DoctorID:** Generate this ID using the Random function for each new doctor.
- **Private string Name:** Take the doctor's name as input.
- **Private string Specialization:** Doctor's specialization (e.g., Cardiologist, Dentist).
- **Private bool IsAvailable:** A flag indicating doctor's availability.



- **Private int GenerateDoctorID():** Generates a unique ID using the Random function.
- **Public Doctor(string name, string specialization):** Assigns values and sets IsAvailable = true.
- **Public int DoctorID:** Getter only.
- **Public string Name:** Getter & Setter.
- **Public string Specialization:** Getter & Setter.
- **Public bool IsAvailable:** Getter.
- **Public void MarkUnavailable():** Sets IsAvailable = false.
- **Public void MarkAvailable():** Sets IsAvailable = true.
- **Override ToString():** Custom string representation of doctor.

Class Patient

- **Private string Name**
- **Private string CNIC**
- **Private List<int> Appointments:** Stores Appointment IDs booked by this patient.
- **Public Patient(string name, string cnic)**
- **Public string CNIC:** Getter only.
- **Public bool HasAppointment(int appointmentID):** Checks if patient already has that appointment.

Class Appointment

- **Private int AppointmentID:** Generate randomly.
- **Private int DoctorID**
- **Private string PatientCNIC**
- **Private DateTime AppointmentDate**
- **Public Appointment(int doctorID, string patientCNIC, DateTime date):** Creates new appointment.
- **Public int AppointmentID:** Getter only.
- **Public override ToString():** Returns formatted appointment details

Class HospitalSystem

- **Private List<Doctor> doctors**
- **Private List<Patient> patients**
- **Private List<Appointment> appointments**
- **Public HospitalSystem()**
- **Public AddPatient():** Takes input, inserts into database & file system.
- **Public UpdatePatient(string cnic):** Updates details in DB & file system.
- **Public DeletePatient(string cnic):** Removes patient from DB & file system.
- **Public DisplayPatients():** Displays all patients.
- **Public AddDoctor():** Inserts doctor into DB & file system.
- **Public UpdateDoctor(int doctorID):** Updates doctor details in DB & file system.



- **Public DeleteDoctor(int doctorID):** Deletes doctor from DB & file system.
- **Public DisplayDoctors():** Displays all doctors with availability.
- **Public BookAppointment(int doctorID, string cnic, DateTime date):** Creates an appointment if doctor is available.
- **Public CancelAppointment(int appointmentID, string cnic):** Cancels an appointment.
- **Public DeclareMostConsultedDoctor():** Displays doctor with maximum appointments.

Serialization & Deserialization

Patient Logging

- Added: When a new patient record is added, it should be serialized and saved in Patient.txt with the prefix "Added:" before the record.
- Deleted: When a patient record is deleted, it should be serialized and saved in Patient.txt with the prefix "Deleted:" before the record.

Doctor Logging

- Added: When a new doctor record is added, it should be serialized and saved in Doctor.txt with the prefix "Added:" before the record.
- Deleted: When a doctor record is deleted, it should be serialized and saved in Doctor.txt with the prefix "Deleted:" before the record.

Appointment Logging

- Added: When a new appointment is made, it should be serialized and saved in Appointment.txt with the prefix "Added:" before the record.
- Deleted: When an appointment is deleted/cancelled, it should be serialized and saved in Appointment.txt with the prefix "Deleted:" before the record.

Database Schemas

Doctor Table

- DoctorID (Primary Key, int)
- Name (nvarchar)
- Specialization (nvarchar)
- IsAvailable (bit)

Patient Table

- PatientID (Primary Key, int, Auto Increment)



- Name (nvarchar)
- CNIC (nvarchar, unique)

Appointment Table

- AppointmentID (Primary Key, int)
 - DoctorID (Foreign Key, int)
 - PatientCNIC (Foreign Key, nvarchar)
 - AppointmentDate (datetime)
-

Menu-based Interface:

Create a menu-driven program that allows the **Admin** to perform all CRUD operations.

1. AddPatient()
2. UpdatePatient(string cnic)
3. DeletePatient(string cnic)
4. DisplayPatients()
5. AddDoctor()
6. UpdateDoctor(int doctorID)
7. DeleteDoctor(int doctorID)
8. DisplayDoctors()
9. BookAppointment(int doctorID, string cnic, DateTime date)
10. CancelAppointment(int appointmentID, string cnic)
11. DeclareMostConsultedDoctor()